```cpp
//LAB CODE BAD MED
// JASON JOHNSON
// JOHNSON JASON
// 4/20/26
// BAD JASON

#pragma once
#include <iostream>
using namespace std;

class FeetInches
{
private:
    int feet;        // To hold a number of feet
    int inches;      // To hold a number of inches
    void simplify(); // Defined in FeetInches.cpp
public:
    // Constructor
    FeetInches(int f = 0, int i = 0);

    // Mutator functions
    void setFeet(int f)
    {
        feet = f;
    }

    void setInches(int i);


    // Accessor functions
    int getFeet() const
    {
        return feet;
    }

    int getInches() const
    {
        return inches;
    }

    // Overloaded operator functions
    FeetInches operator + (const FeetInches&); // Overloaded +
    FeetInches operator - (const FeetInches&); // Overloaded -
    FeetInches operator ++ ();              // Prefix ++
    FeetInches operator ++ (int);           // Postfix ++
    bool operator > (const FeetInches&);      // Overloaded >
    bool operator < (const FeetInches&);      // Overloaded <
    bool operator == (const FeetInches&);     // Overloaded ==
    // Friends
```

```
    friend ostream& operator << (ostream&, const FeetInches&);
    friend istream& operator >> (istream&, FeetInches&);
};

void FeetInches::simplify() {
    int totalInches = (feet * 12) + inches;
    feet = totalInches / 12;
    inches = totalInches % 12;

}


FeetInches::FeetInches(int f, int i) {
    feet = f;
    inches = i;

}

void FeetInches::setInches(int i) {
    inches = i;
}



 FeetInches operator + (const FeetInches& first, FeetInches& second) {
     return FeetInches(first.inches + second.getInches);

}

FeetInches operator - (const FeetInches& first, FeetInches& second) {
    if (*this <) {

    }
}

  FeetInches operator ++ () {
     return inches++;
}

FeetInches operator ++ (int i) {
    return inches++;
}

bool operator > (const FeetInches& first, const FeetInches& second) {
    return first > second;
}

bool operator < (const FeetInches& first, FeetInches& second ) {
    return first < second;
```

```
}


bool operator == (const FeetInches& first, FeetInches& second) {
    return first == second;
}

// Friends
ostream& operator << (ostream& stream, const FeetInches& f) {
    stream <<
 }

istream& operator >> (istream& stream, FeetInches& f) {


 }



























// This program demonstrates the << and >> operators,
// overloaded to work with the FeetInches class.


int main()
{
```

```cpp
    FeetInches first, second;  // Define two objects.

    // Get a distance for the first object.
    cout << "Enter a distance in feet and inches.\n";
    cin >> first;

    // Get a distance for the second object.
    cout << "Enter another distance in feet and inches.\n";
    cin >> second;

    // Display the values in the objects.
    cout << "\nThe values you entered are:\n\n";

    cout << first << " and " << second << endl << endl;

    if (first > second)
    {
        cout << "First is longer than the second.\n\n";
    }
    else
    {
        cout << "The second is longer than the first.\n\n";
    }


    if (first < second)
    {
        cout << "The Second is longer than the second.\n\n";
    }
    else
    {
        cout << "The First is longer than the first.\n\n";
    }

    if (first == second)
    {
        cout << "Both lengths are the same\n\n";
    }
    else
    {
        cout << "length are not the same.\n\n";

    }


    cout << "Adding first to second " << first + second << endl << endl;;

    cout << "Subtracting first from second " << first - second << endl << 
      endl;
```

```

    cout << "Incrementing first " << ++first << endl << endl;

    cout << "Incrementing second " << ++second << endl << endl;

    cout << "Incrementing first " << first++ << endl << endl;

    cout << "Incrementing second " << second++ << endl << endl;



    return 0;
}



```